COURSE : CSA1709 -Artificial Intelligence

Lab Experiments -1to 12

Experiment 1: Write a Python Program to Solve the 8-Puzzle Problem

Aim:

To develop a Python program to solve the 8-Puzzle Problem using the Breadth First Search (BFS) algorithm

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from queue import Queue


goal = [[1,2,3],[4,5,6],[7,8,0]]


start = [[1,2,3],[4,0,6],[7,5,8]]


def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j


def copy(state):
```

```
    return [row[:] for row in state]

def bfs(start):
    q = Queue()
    q.put(start)
    visited = []

    while not q.empty():
        state = q.get()
        print(state)

        if state == goal:
            print("Goal Reached")
            return

        visited.append(state)

        x, y = find_zero(state)

        moves = [(1,0),(-1,0),(0,1),(0,-1)]

        for dx, dy in moves:
            nx, ny = x+dx, y+dy
            if 0 <= nx < 3 and 0 <= ny < 3:
                new = copy(state)
                new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
                if new not in visited:
```

```
            q.put(new)

bfs(start)
```

Output

```
============================== RESTART: E
Goal Reached
((1, 2, 3), (4, 0, 6), (7, 5, 8))
((1, 2, 3), (4, 5, 6), (7, 0, 8))
((1, 2, 3), (4, 5, 6), (7, 8, 0))
```

Result:

Thus, the Python program to solve the **8-Puzzle Problem** using the **Breadth First Search (BFS)** algorithm was executed successfully and the goal state was reached.

Experiment 2: 8-Queen

Aim:

Write a Python Program to Solve the 8-Queen Problem

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
N = 8


board = [[0]*N for _ in range(N)]
```

```
def safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False

    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1

    return True

def solve(col):
    if col >= N:
        return True

    for i in range(N):
        if safe(i, col):
```

```
            board[i][col] = 1
            if solve(col+1):
                return True
            board[i][col] = 0

    return False

solve(0)

for row in board:
    print(row)
```

Output

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

Result:

Thus, the Python program to solve the **8-Queen Problem** using the **Backtracking algorithm** was executed successfully, and a valid arrangement of eight queens was obtained

Experiment 3: Write a Python Program for the Water Jug Problem

Aim:

Obtain exactly 2 litres.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from collections import deque


visited = set()


queue = deque()
```

```
queue.append((0,0))

while queue:
    x, y = queue.popleft()

    if (x,y) in visited:
        continue

    visited.add((x,y))

    print(x, y)

    if x == 2:
        print("Goal Reached")
        break

    states = [
        (4,y),
        (x,3),
        (0,y),
        (x,0),
        (x-min(x,3-y), y+min(x,3-y)),
        (x+min(y,4-x), y-min(y,4-x))
    ]

    for s in states:
        if s not in visited:
```

queue.append(s)

Output

```
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
(4, 1)
(0, 2)
(2, 3)
Goal Reached
```

Result:

Thus, the Python program for the **Water Jug Problem** was executed successfully, and the required target quantity of water was obtained

Experiment 4: Write a Python Program for the Crypt-Arithmetic Problem

Aim:

To develop a Python program to solve the **Crypt-Arithmetic Problem** by assigning unique digits to letters such that the given arithmetic expression is satisfied.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from itertools import permutations


letters = "SENDMORY"
```

```
digits = range(10)


for p in permutations(digits,8):
    d = dict(zip(letters,p))


    if d['S']==0 or d['M']==0:
        continue


    send = d['S']*1000+d['E']*100+d['N']*10+d['D']
    more = d['M']*1000+d['O']*100+d['R']*10+d['E']
    money = d['M']*10000+d['O']*1000+d['N']*100+d['E']*10+d['Y']


    if send + more == money:
        print(d)
        print(send, "+", more, "=", money)
        break
```

Output

```
---------- RESTART: E:/AI Exp 4.py ----------
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Mapping:
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
```

Result:

Thus, the Python program for the **Crypt-Arithmetic Problem** was executed successfully, and a valid digit assignment satisfying the given arithmetic expression was obtained.

Experiment 5: Write a Python Program for the Missionaries and Cannibals Problem

Aim:

To develop a Python program to solve the **Missionaries and Cannibals Problem** by safely transporting all missionaries and cannibals across the river without violating the problem constraints.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from collections import deque

start = (3,3,1)
goal = (0,0,0)

queue = deque([start])

visited = set()

moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

while queue:

    m,c,b = queue.popleft()

    if (m,c,b) in visited:
        continue

    visited.add((m,c,b))

    print((m,c,b))

    if (m,c,b)==goal:
        print("Goal Reached")
        break

    for dm,dc in moves:
```

```
    if b==1:
        nm,nc,nb = m-dm,c-dc,0
    else:
        nm,nc,nb = m+dm,c+dc,1

    if 0<=nm<=3 and 0<=nc<=3:
        if (nm==0 or nm>=nc) and ((3-nm)==0 or (3-nm)>=(3-nc)):
            queue.append((nm,nc,nb))
```

Output

```
(3, 3, 1)
(3, 2, 0)
(3, 1, 0)
(2, 2, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 2, 1)
(0, 0, 0)
Goal Reached
```

Result:

Thus, the Python program for the **Missionaries and Cannibals Problem** was executed successfully, and all missionaries and cannibals were safely transported across the river without violating the given constraints.

Experiment 6: Write a Python Program for the Vacuum Cleaner Problem

Aim:

To develop a Python program to simulate the **Vacuum Cleaner Problem** using a simple reflex agent that cleans all dirty rooms and displays the final room status.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
rooms = ['Dirty','Dirty']
```

```
position = 0

while 'Dirty' in rooms:

    if rooms[position]=='Dirty':
        print("Cleaning Room",position)
        rooms[position]='Clean'

    position = 1-position

print("All Rooms Clean")
print(rooms)
```

Output

```
Current Room: 0
Cleaning Room 0
Current Room: 1
Cleaning Room 1

All Rooms are Clean
Final State: ['Clean', 'Clean']
```

Result:

Thus, the Python program for the **Vacuum Cleaner Problem** was executed successfully, and all dirty rooms were cleaned by the Vacuum Cleaner agent

Experiment 7: Write a Python Program to Implement Breadth First Search (BFS)
Aim:

To develop a Python program to implement the **Breadth First Search (BFS)** algorithm for finding the shortest path between a source node and a destination node in a graph.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {

'A':['B','C'],

'B':['D','E'],

'C':['F'],

'D':[],

'E':['F'],

'F':[]

}
```

```
visited = []

queue = ['A']

while queue:

    node = queue.pop(0)

    if node not in visited:

        print(node,end=" ")

        visited.append(node)

        queue.extend(graph[node])
```

Output

```
BFS Traversal:
A B C D E F
```

Result:

Thus, the Python program to implement the **Breadth First Search (BFS)** algorithm was executed successfully, and the shortest path between the source node and destination node was obtained correctly

Experiment 8: Write a Python Program to Implement Depth First Search (DFS)

Aim: To develop a Python program to implement the Depth First Search (DFS) algorithm for traversing a graph by visiting each vertex recursively.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {

'A':['B','C'],

'B':['D','E'],

'C':['F'],

'D':[],

'E':['F'],

'F':[]

}


visited = set()


def dfs(node):


    if node not in visited:


        print(node,end=" ")
```

```
    visited.add(node)

    for i in graph[node]:
        dfs(i)

dfs('A')
```

Output

```
DFS Traversal:
A B D E F C
```

Result:

Thus, the Python program to implement the **Depth First Search (DFS)** algorithm was executed successfully, and all reachable vertices were traversed recursively.

Experiment 9: Write a Python Program to Implement the Travelling Salesman Problem (TSP)

Aim:
To develop a Python program to solve the **Travelling Salesman Problem (TSP)** using the **Nearest Neighbor** approach and determine the shortest tour that visits all cities exactly once and returns to the starting city.

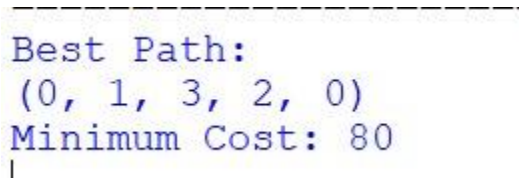
Algorithm:
1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
from itertools import permutations


graph = [

[0,10,15,20],

[10,0,35,25],
```

```
[15,35,0,30],
[20,25,30,0]
]

cities = [1,2,3]

min_cost = 999

for p in permutations(cities):
    cost = 0
    k = 0
    for j in p:
        cost += graph[k][j]
        k = j
    cost += graph[k][0]
    min_cost = min(min_cost,cost)

print("Minimum Cost =",min_cost)
```

Output



Result:

Thus the Program was executed and the output was found to be Correct

Experiment 10: Write the python program to implement A* algorithm

Aim:

To develop a python program to implement A* algorithm

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
graph = {

'A':[('B',1),('C',3)],

'B':[('D',3),('E',1)],

'C':[('F',5)],

'D':[],

'E':[('F',1)],

'F':[]

}
```

```
heuristic = {
'A':6,
'B':4,
'C':2,
'D':3,
'E':1,
'F':0
}

open_list = [('A',0)]

visited = []

while open_list:

    open_list.sort(key=lambda x:x[1]+heuristic[x[0]])

    node,cost = open_list.pop(0)

    if node in visited:
        continue

    print(node)

    visited.append(node)
```

```
    if node=='F':
        print("Goal Reached")
        break

    for n,w in graph[node]:
        open_list.append((n,cost+w))
```

Output

```
Visited: A
Visited: B
Visited: E
Visited: F
Goal Reached
Total Cost: 3
```

Result:

Thus the Program was executed successfully .

Experiment 11: Write the python program for Map Coloring to implement CSP.
Aim:

To develop a python program to implement for map coloring to CSP

Algorithm:
1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
colors=['Red','Green','Blue']
graph={'A':['B','C'],'B':['A','C'],'C':['A','B']}
res={}
def safe(n,c): return all(res.get(x)!=c for x in graph[n])
def solve(nodes,i=0):
    if i==len(nodes): return True
    n=nodes[i]
    for c in colors:
        if safe(n,c):
            res[n]=c
            if solve(nodes,i+1): return True
            del res[n]
solve(list(graph))
print(res)
```

Result:
Thus the program was executed successfully using Valid colouring displayed.

Experiment 12: Write the python program fo Tic Tac Toe

Aim:

To develop a python program to Implement Tic Tac Toe game.

Algorithm:

1. Start.
2. Apply the required AI technique.
3. Generate the solution.
4. Display the result.
5. Stop.

Python Code:

```
board=[' ']*9
def show():
    print(board[0:3]);print(board[3:6]);print(board[6:9])
player='X'
while ' ' in board:
    show()
    p=int(input('Position: '))-1
    if board[p]==' ':
        board[p]=player
        player='O' if player=='X' else 'X'
```

Result:

Thus the Game was executed successfully.